

Что такое строка в Go

В Go строка (string) представляет собой неизменяемую последовательность байтов. Строки в Go имеют тип `string` и могут содержать любые данные, в том числе UTF-8 символы. Под капотом строки представляют собой структуру, содержащую указатель на массив байтов и длину строки.

Объявление строк

Строки можно объявлять различными способами:

```
// Объявление строки с помощью двойных кавычек
var str1 string = "Hello, World!"
```

```
// Краткая форма объявления
str2 := "Hello, Go!"
```

```
// Многострочные строки с помощью обратных кавычек
str3 := `This is a
multiline string`
```

Манипуляция над строками

Так как строки в Go неизменяемы, большинство операций над строками создают новые строки.

- Конкатенация:

```
s1 := "Hello, "
s2 := "World!"
s3 := s1 + s2 // "Hello, World!"
```

- Доступ к символам (байтам):

```
s := "Hello"
c := s[1] // 'e' (байт с кодом 101)
```

- Извлечение подстроки:

```
s := "Hello, World!"
sub := s[7:12] // "World"
```

Сравнение строк

Сравнение строк основано на сравнении последовательностей байтов. Если первая строка больше, чем вторая, то возвращается значение больше нуля. В Go строки можно сравнивать с помощью операторов `==`, `!=`, `<`, `>`, `<=`, `>=`. Сравнение происходит в лексикографическом порядке:

```
s1 := "abc"
s2 := "def"
fmt.Println(s1 == s2) // false
fmt.Println(s1 < s2)  // true
```

Поиск подстроки в строке

Для поиска подстроки в строке используется функция `strings.Contains` и другие методы из пакета `strings`:

```
import "strings"

s := "Hello, World!"
fmt.Println(strings.Contains(s, "World")) // true
fmt.Println(strings.Index(s, "World"))    // 7 (начальный индекс)
```

Руны и Байты в Go

Байты

Байт (byte) – это базовая единица данных в компьютере, представляющая собой 8 бит. В Go `byte` является алиасом для типа `uint8`. Байты часто используются для работы с низкоуровневыми данными, такими как сжатие данных, сетевые протоколы и кодировка символов.

Руны

Руна (rune) – это особый тип данных в Go. Руна представляет собой Unicode символ. Руны часто используются для обработки и обработки текстовых данных. Руна (rune) – это алиас для типа `int32` и представляет собой Unicode кодовую точку. Использование рун позволяет работать с символами Unicode, каждый из которых может занимать более одного байта.

```
package main

import (
    "fmt"
)

func main() {
    s := "Hello, 世界"

    // Преобразование строки в байты
    bytes := []byte(s)
    fmt.Println(bytes) // [72 101 108 108 111 44 32 228 184 150 231 149 140]

    // Преобразование строки в руны
    runes := []rune(s)
    fmt.Println(runes) // [72 101 108 108 111 44 32 19990 30028]
}
```

ASCII и UTF-8

ASCII

ASCII (American Standard Code for Information Interchange) – это стандарт кодирования символов, использующий 7 бит для представления символов. Это позволяет закодировать 128 различных символов, включая латинские буквы, цифры и некоторые управляющие символы. ASCII является простым и широко используемым стандартом, но он ограничен в количестве символов и не подходит для представления всех символов в других языках.

Пример ASCII-кодов:

- 'A' = 65
- 'B' = 66
- 'a' = 97
- 'b' = 98

UTF-8

UTF-8 (Unicode Transformation Format 8-bit) – это переменная длина кодировки символов Unicode, использующая от 1 до 4 байт для представления каждого символа. UTF-8 является обратно совместимой с ASCII и может представлять любой символ Unicode. UTF-8 является наиболее распространенной кодировкой в Интернете благодаря своей гибкости и эффективности.

Пример UTF-8-кодов:

- 'A' = 0b01000001
- 'B' = 0b01000010
- 'a' = 0b01100001
- 'b' = 0b01100010
- Символы ASCII занимают 1 байт.
- Символы, такие как 'é' (U+00E9), занимают 2 байта.
- Символы, такие как '世' (U+4E16), занимают 3 байта.
- Символы, такие как '☺' (U+1F600), занимают 4 байта.

Зачем они используются

Байты

Байты используются для низкоуровневых операций с данными, таких как работа с бинарными файлами, сетевыми протоколами и сжатие данных. Они также являются основой для представления строк в памяти.

Руны

Руны используются для работы с символами Unicode, что позволяет поддерживать многоязычные тексты и специальные символы. Это важно для глобальных приложений, которые должны работать с текстами на разных языках.

ASCII

ASCII используется для представления текстов на английском языке и других текстов, которые можно выразить ограниченным набором символов. Его простота и эффективность делают его полезным для простых текстовых данных и систем с ограниченными ресурсами.

UTF-8

UTF-8 является стандартом де-факто для кодирования текстов в Интернете и других системах, где важна поддержка многоязычных текстов и совместимость с существующими системами.

```

package main

import (
    "fmt"
    "unicode/utf8"
)

func main() {
    s := "Hello, 世界"

    // Длина строки в байтах
    fmt.Println(len(s)) // 13

    // Длина строки в рунах (символах)
    fmt.Println(utf8.RuneCountInString(s)) // 9

    // Итерация по рунам в строке
    for i, r := range s {
        fmt.Printf("%d: %c\n", i, r)
    }
}

```

Этот пример показывает, как строка "Hello, 世界" занимает 13 байт, но содержит 9 символов (рун). Итерация по строке с использованием `range` позволяет корректно обрабатывать каждый символ Unicode независимо от его длины в байтах.

Как работает UTF-8

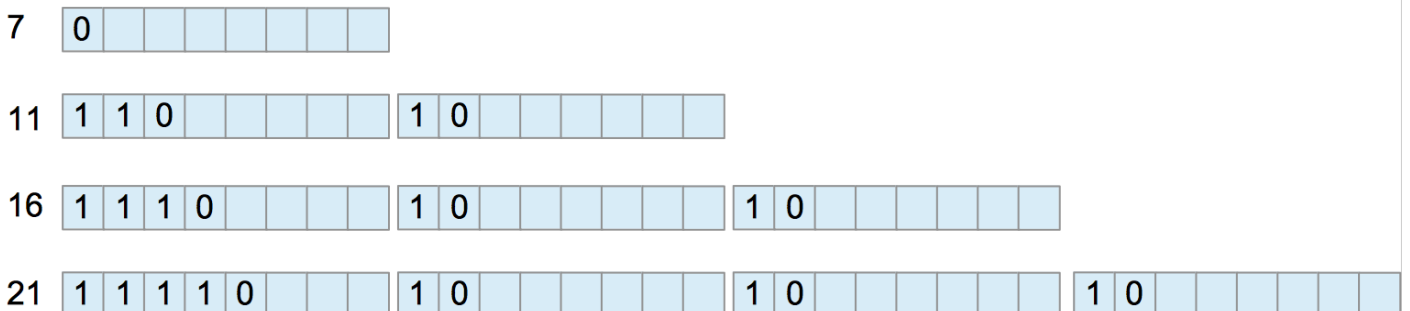
UTF-8 (Unicode Transformation Format - 8-bit) – это переменная длина кодировки символов Unicode, использующая от 1 до 4 байт для представления каждого символа. UTF-8 разработан для обратной совместимости с ASCII, где первые 128 кодовых точек (0-127) закодированы как один байт, что делает его очень эффективным для текста, содержащего только латинские символы.

Структура UTF-8

Каждый байт в UTF-8 имеет определенную структуру, которая указывает, сколько байтов занимает символ:

- 1 байт: 0xxxxxxx
- 2 байта: 110xxxxx 10xxxxxx
- 3 байта: 1110xxxx 10xxxxxx 10xxxxxx
- 4 байта: 11110xxx 10xxxxxx 10xxxxxx 10xxxxxx

Bits free



Для понимания, давайте рассмотрим несколько примеров кодировки символов в UTF-8:

- ASCII символ A (U+0041): 01000001 (1 байт)
- Латинская буква с диакритическим знаком é (U+00E9): 11000011 10101001 (2 байта)
- Китайский иероглиф 世 (U+4E16): 11100100 10111000 10010110 (3 байта)
- Эмодзи 😊 (U+1F600): 11110000 10011111 10011000 10000000 (4 байта)

Определение длины символа

Чтобы определить, сколько байтов занимает символ в UTF-8, можно использовать следующий алгоритм:

- Прочитать первый байт.
- Если первый байт начинается с 0, символ занимает 1 байт.
- Если первый байт начинается с 110, символ занимает 2 байта.
- Если первый байт начинается с 1110, символ занимает 3 байта.
- Если первый байт начинается с 11110, символ занимает 4 байта.

```
package main

import (
    "fmt"
    "unicode/utf8"
)

func main() {
    // Пример строки с различными символами
    str := "Aé世😊"

    for len(str) > 0 {
        // Получаем первый байт
        r, size := utf8.DecodeRuneInString(str)

        fmt.Printf("Руна: %c, Unicode: U+%04X, Занимает %d байт\n", r, r, size)

        // Обрезаем строку, удаляя первый символ
        str = str[size:]
    }
}
```

Руна: A, Unicode: U+0041, Занимает 1 байт
Руна: é, Unicode: U+00E9, Занимает 2 байта
Руна: 世, Unicode: U+4E16, Занимает 3 байта
Руна: 😊, Unicode: U+1F600, Занимает 4 байта

Кодировка символа по байтам

Для декодирования символа из набора байтов в Go можно использовать функции из пакета `unicode/utf8`:

```

package main

import (
    "fmt"
    "unicode/utf8"
)

func main() {
    // Пример байтовой последовательности для символов "Aé世☺"
    data := []byte{0x41, 0xC3, 0xA9, 0xE4, 0xB8, 0x96, 0xF0, 0x9F, 0x98, 0x80}

    for len(data) > 0 {
        // Декодируем первую руну
        r, size := utf8.DecodeRune(data)

        fmt.Printf("Руна: %c, Unicode: U+%04X, Занимает %d байт\n", r, r, size)

        // Обрезаем байтовый массив, удаляя первый символ
        data = data[size:]
    }
}

```

Этот код декодирует байты обратно в символы и показывает, сколько байтов занимает каждый символ.

Заключение

UTF-8 – это эффективная и гибкая кодировка, позволяющая представлять все символы Unicode, при этом сохраняя обратную совместимость с ASCII. Использование UTF-8 позволяет эффективно хранить и обрабатывать тексты на различных языках, что делает её идеальной для глобальных приложений и веб-технологий.

Пакеты для работы со строками

Пакет strings

Пакет `strings` предоставляет множество полезных функций для работы со строками:

- `strings.Contains(s, substr string) bool`
- `strings.Index(s, substr string) int`
- `strings.ToUpper(s string) string`
- `strings.ToLower(s string) string`
- `strings.Replace(s, old, new string, n int) string`
- `strings.Split(s, sep string) []string` и т.д.

Contains

Функция `strings.Contains` проверяет, содержится ли подстрока `substr` в строке `s`. Возвращает `true`, если подстрока содержится в строке.

```

import "strings"

s := "Hello, World!"
contains := strings.Contains(s, "World")
fmt.Println(contains) // true

```

Count

Функция `strings.Count` возвращает количество вхождений подстроки `substr` в строку `s`. Если подстрока `substr` не содержится в строке, то функция вернет 0.

```
import "strings"

s := "Hello, Hello, Hello!"
count := strings.Count(s, "Hello")
fmt.Println(count) // 3
```

HasPrefix и HasSuffix

Функция `strings.HasPrefix` проверяет, начинается ли строка `s` с префикса `prefix`. Функция `strings.HasSuffix` проверяет, заканчивается ли строка `s` суффиксом `suffix`.

```
import "strings"

s := "Hello, World!"
hasPrefix := strings.HasPrefix(s, "Hello")
fmt.Println(hasPrefix) // true

hasSuffix := strings.HasSuffix(s, "World!")
fmt.Println(hasSuffix) // true
```

Index и LastIndex

Функция `strings.Index` и `strings.LastIndex` возвращают индекс первого вхождения или последнего вхождения подстроки `substr` в строке `s`. Если подстрока `substr` не содержится в строке, то функция вернет -1.

```
import "strings"

s := "Hello, World!"
index := strings.Index(s, "World")
fmt.Println(index) // 7

lastIndex := strings.LastIndex(s, "l")
fmt.Println(lastIndex) // 10
```

Join

Функция `strings.Join` объединяет срез строк в одну строку с разделителем `sep`.

```
import "strings"

parts := []string{"Hello", "World"}
joined := strings.Join(parts, ", ")
fmt.Println(joined) // "Hello, World"
```

Repeat

Функция `strings.Repeat` повторяет строку `s` заданное количество раз.

```
import "strings"

s := "Go"
repeated := strings.Repeat(s, 3)
fmt.Println(repeated) // "GoGoGo"
```

Replace и ReplaceAll

Функция `strings.Replace` и `strings.ReplaceAll` заменяют все вхождения подстроки `old` в строке `s` на подстроку `new`. Если параметр `n` больше или равен нулю, то функция вернет строку, в которой все вхождения подстроки `old` заменены на подстроку `new`.

```
import "strings"

s := "Hello, World! Hello, Go!"
replaced := strings.Replace(s, "Hello", "Hi", 1)
fmt.Println(replaced) // "Hi, World! Hello, Go!"

replacedAll := strings.ReplaceAll(s, "Hello", "Hi")
fmt.Println(replacedAll) // "Hi, World! Hi, Go!"
```

Split и SplitN

Функция `strings.Split` разделяет строку `s` на подстроки по разделителю `sep`. Если параметр `n` больше или равен нулю, то функция вернет срез с подстроками, разделенными разделителем `sep`. Если параметр `n` меньше или равен нулю, то функция вернет срез с подстроками, разделенными разделителем `sep`, до конца строки.

```
import "strings"

s := "a,b,c,d"
split := strings.Split(s, ",")
fmt.Println(split) // ["a", "b", "c", "d"]

splitN := strings.SplitN(s, ",", 2)
fmt.Println(splitN) // ["a", "b,c,d"]
```

ToLower и ToUpper

Функция `strings.ToLower` возвращает строку `s`, в которой все символы были в нижнем регистре. Функция `strings.ToUpper` возвращает строку `s`, в которой все символы были в верхнем регистре.

```
import "strings"

s := "Hello, World!"
lower := strings.ToLower(s)
fmt.Println(lower) // "hello, world!"

upper := strings.ToUpper(s)
fmt.Println(upper) // "HELLO, WORLD!"
```

Trim, TrimSpace, TrimPrefix и TrimSuffix

Функция `strings.Trim` удаляет пробелы в начале и в конце строки `s`.
Функция `strings.TrimSpace` удаляет пробелы в начале и в конце строки `s`.
Функция `strings.TrimPrefix` удаляет префикс `prefix` из строки `s`.
Функция `strings.TrimSuffix` удаляет суффикс `suffix` из строки `s`.


```
import "strings"

s := " Hello, World! "
trimmed := strings.Trim(s, " ")
fmt.Println(trimmed) // "Hello, World!"

trimSpace := strings.TrimSpace(s)
fmt.Println(trimSpace) // "Hello, World!"

prefix := "Hello"
suffix := "World!"
trimPrefix := strings.TrimPrefix(s, prefix)
fmt.Println(trimPrefix) // " Hello, World! "

trimSuffix := strings.TrimSuffix(s, suffix)
fmt.Println(trimSuffix) // " Hello, World! "
```

Заключение

Пакет `strings` предоставляет множество функций для работы со строками в Go. Эти функции делают обработку строк простой и эффективной, помогая решать многие распространенные задачи, такие как поиск, замена, разделение и объединение строк.

Пакет `strconv`

Пакет `strconv` в Go предоставляет функции для преобразования данных между строками и базовыми типами (например, целыми числами, числами с плавающей запятой и булевыми значениями)

- `strconv.Atoi(s string) (int, error)` – строка в целое число
- `strconv.Itoa(i int) string` – целое число в строку
- `strconv.ParseFloat(s string, bitSize int) (float64, error)` – строка в число с плавающей запятой
- `strconv.FormatFloat(f float64, fmt byte, prec, bitSize int) string` – число с плавающей запятой в строку

Atoi и Itoa

Функция `strconv.Atoi` преобразует строку в целое число. Если строка не может быть преобразована в целое число, то функция вернет ошибку. Функция `strconv.Itoa` преобразует целое число в строку.

```
import (
    "fmt"
    "strconv"
)

func main() {
    // Преобразование строки в целое число
    i, err := strconv.Atoi("123")
    if err != nil {
        fmt.Println(err)
    } else {
        fmt.Println(i) // 123
    }

    // Преобразование целого числа в строку
    s := strconv.Itoa(123)
    fmt.Println(s) // "123"
}
```

ParseInt и FormatInt

Функция `strconv.ParseInt` преобразует строку в целое число. Если строка не может быть преобразована в целое число, то функция вернет ошибку. Функция `strconv.FormatInt` преобразует целое число в строку.

```
import (
    "fmt"
    "strconv"
)

func main() {
    // Преобразование строки в целое число с указанием базы
    i, err := strconv.ParseInt("1A", 16, 64)
    if err != nil {
        fmt.Println(err)
    } else {
        fmt.Println(i) // 26
    }

    // Преобразование целого числа в строку с указанием базы
    s := strconv.FormatInt(26, 16)
    fmt.Println(s) // "1a"
}
```

ParseFloat и FormatFloat

Функция `strconv.ParseFloat` преобразует строку в число с плавающей запятой. Если строка не может быть преобразована в число с плавающей запятой, то функция вернет ошибку. Функция `strconv.FormatFloat` преобразует число с плавающей запятой в строку.

```
import (
    "fmt"
    "strconv"
)

func main() {
    // Преобразование строки в число с плавающей запятой
    f, err := strconv.ParseFloat("3.14159", 64)
    if err != nil {
        fmt.Println(err)
    } else {
        fmt.Println(f) // 3.14159
    }

    // Преобразование числа с плавающей запятой в строку
    s := strconv.FormatFloat(3.14159, 'f', 2, 64)
    fmt.Println(s) // "3.14"
}
```

ParseBool и FormatBool

Функция `strconv.ParseBool` преобразует строку в булево значение. Если строка не может быть преобразована в булево значение, то функция вернет ошибку. Функция `strconv.FormatBool` преобразует булево значение в строку.

```
import (
    "fmt"
    "strconv"
)

func main() {
    // Преобразование строки в булево значение
    b, err := strconv.ParseBool("true")
    if err != nil {
        fmt.Println(err)
    } else {
        fmt.Println(b) // true
    }

    // Преобразование булевого значения в строку
    s := strconv.FormatBool(true)
    fmt.Println(s) // "true"
}
```

Quote и Unquote

Функция `strconv Quote` преобразует строку в строку с экранированными символами.
 Функция `strconv Unquote` преобразует строку с экранированными символами в строку.

```
import (
    "fmt"
    "strconv"
)

func main() {
    // Добавление кавычек к строке
    s := strconv.Quote("Hello, World!")
    fmt.Println(s) // "\"Hello, World!\""

    // Удаление кавычек из строки
    unquoted, err := strconv.Unquote(s)
    if err != nil {
        fmt.Println(err)
    } else {
        fmt.Println(unquoted) // "Hello, World!"
    }
}
```

Append функции

Функция `strconv Append` преобразует целое число в строку с указанием базы.
 Функция `strconv AppendInt` преобразует целое число в строку с указанием базы.
 Функция `strconv AppendFloat` преобразует число с плавающей запятой в строку.
 Функция `strconv AppendBool` преобразует булево значение в строку.

```
import (
    "fmt"
    "strconv"
)

func main() {
    b := []byte("Number: ")
    b = strconv.AppendInt(b, 123, 10)
    fmt.Println(string(b)) // "Number: 123"
}
```

Заключение

Пакет `strconv` предоставляет мощные инструменты для преобразования данных между строками и базовыми типами. Эти функции являются неотъемлемой частью многих программ на Go, позволяя легко работать с данными из внешних источников, форматировать и обрабатывать ввод пользователя, а также выполнять другие задачи, требующие преобразования типов.

Пакет `unicode`

Пакет `unicode` в Go предоставляет функции для работы с символами Unicode и их свойствами. Он особенно полезен для обработки текстов, содержащих символы различных языков и специальных символов.

`IsLetter`

Функция `unicode.IsLetter` проверяет, является ли символ буквой. Если символ является буквой, функция вернет `true`, в противном случае вернет `false`.

```
import (
    "fmt"
    "unicode"
)

func main() {
    r := 'A'
    fmt.Println(unicode.IsLetter(r)) // true

    r = '1'
    fmt.Println(unicode.IsLetter(r)) // false
}
```

`IsDigit`

Функция `unicode.IsDigit` проверяет, является ли символ цифрой. Если символ является цифрой, функция вернет `true`, в противном случае вернет `false`.

```
import (
    "fmt"
    "unicode"
)

func main() {
    r := '1'
    fmt.Println(unicode.IsDigit(r)) // true

    r = 'A'
    fmt.Println(unicode.IsDigit(r)) // false
}
```

`IsSpace`

Функция `unicode.IsSpace` проверяет, является ли символ пробелом. Если символ является пробелом, функция вернет `true`, в противном случае вернет `false`.

```
import (
    "fmt"
    "unicode"
)

func main() {
    r := ' '
    fmt.Println(unicode.IsSpace(r)) // true

    r = '\t'
    fmt.Println(unicode.IsSpace(r)) // true

    r = 'A'
    fmt.Println(unicode.IsSpace(r)) // false
}
```

IsUpper и IsLower

Функция `unicode.IsUpper` проверяет, является ли символ прописной. Если символ является прописной, функция вернет `true`, в противном случае вернет `false`.

```
import (
    "fmt"
    "unicode"
)

func main() {
    r := 'A'
    fmt.Println(unicode.IsUpper(r)) // true

    r = 'a'
    fmt.Println(unicode.IsLower(r)) // true
}
```

ToUpper и ToLower

Функция `unicode.ToUpper` преобразует символ в прописную букву. Функция `unicode.ToLower` преобразует символ в строчную букву.

```
import (
    "fmt"
    "unicode"
)

func main() {
    r := 'a'
    fmt.Println(string(unicode.ToUpper(r))) // "A"

    r = 'A'
    fmt.Println(string(unicode.ToLower(r))) // "a"
}
```

Is функции для различных категорий символов

Функция `unicode.IsDigit` проверяет, является ли символ цифрой. Если символ является цифрой, функция вернет `true`, в противном случае вернет `false`. Пакет `unicode` содержит множество функций для проверки различных категорий символов, таких как `IsControl`, `IsPunct`, `IsSymbol` и другие.

```
import (
    "fmt"
    "unicode"
)

func main() {
    r := '@'
    fmt.Println(unicode.IsPunct(r)) // true

    r = '\n'
    fmt.Println(unicode.IsControl(r)) // true

    r = '$'
    fmt.Println(unicode.IsSymbol(r)) // true
}
```

In функция для проверки принадлежности символа к конкретному набору

Функция `unicode.In` проверяет, принадлежит ли символ набору. Если символ принадлежит набору, функция вернет `true`, в противном случае вернет `false`. Пакет `unicode` содержит множество функций для проверки принадлежности символа к конкретному набору.

```
import (
    "fmt"
    "unicode"
    "unicode/latin1"
)

func main() {
    r := 'Æ'
    fmt.Println(unicode.In(r, unicode.Latin, latin1.Supplement)) // true
}
```

Работа с диапазонами символов

Для работы с диапазонами символов в Go можно использовать функцию `unicode.Range`. Пакет `unicode` также предоставляет поддержку для работы с диапазонами символов. Эти диапазоны определены в виде переменных, таких как `unicode.Letter`, `unicode.Digit` и т.д.

Заключение

Пакет `unicode` предоставляет мощные инструменты для работы с символами Unicode в Go. Он позволяет определять свойства символов, такие как принадлежность к категориям, регистр, и преобразовывать символы между регистрами. Эти функции делают обработку текстов на различных языках удобной и эффективной, что особенно важно для глобальных приложений.

Пакет utf8

Пакет `utf8` в Go предоставляет функции для работы с UTF-8 кодировкой, которая является стандартной кодировкой символов Unicode в Go. Давайте рассмотрим основные функции пакета `utf8` и примеры их использования.

DecodeRune и DecodeRuneInString

Функции `DecodeRune` и `DecodeRuneInString` используются для декодирования первого символа UTF-8 из среза байтов или строки соответственно.

```
import (
    "fmt"
    "unicode/utf8"
)

func main() {
    b := []byte("Hello, 世界")
    r, size := utf8.DecodeRune(b)
    fmt.Printf("Rune: %c, Size: %d\n", r, size) // Rune: H, Size: 1

    s := "Hello, 世界"
    r, size = utf8.DecodeRuneInString(s)
    fmt.Printf("Rune: %c, Size: %d\n", r, size) // Rune: H, Size: 1
}
```

EncodeRune

Функция `EncodeRune` кодирует символ Unicode в UTF-8 и записывает его в срез байтов.

```
import (
    "fmt"
    "unicode/utf8"
)

func main() {
    r := '世'
    buf := make([]byte, 3)
    size := utf8.EncodeRune(buf, r)
    fmt.Printf("Encoded: % x, Size: %d\n", buf, size) // Encoded: e4 b8 96, Size: 3
}
```

RuneLen

Функция `RuneLen` возвращает количество байтов, необходимых для кодирования символа Unicode в UTF-8.

```
import (
    "fmt"
    "unicode/utf8"
)

func main() {
    r := '世'
    size := utf8.RuneLen(r)
    fmt.Printf("Rune: %c, Size: %d\n", r, size) // Rune: 世, Size: 3
}
```

RuneCount и RuneCountInString

Функции `RuneCount` и `RuneCountInString` возвращают количество символов Unicode (рун) в строке или срезе байтов.

```
import (
    "fmt"
    "unicode/utf8"
)

func main() {
    b := []byte("Hello, 世界")
    count := utf8.RuneCount(b)
    fmt.Printf("Byte slice: %d runes\n", count) // Byte slice: 9 runes

    s := "Hello, 世界"
    count = utf8.RuneCountInString(s)
    fmt.Printf("String: %d runes\n", count) // String: 9 runes
}
```

Valid и ValidString

Функции `Valid` и `ValidString` возвращают `true`, если строка состоит только из символов Unicode и `false` в противном случае.

```
import (
    "fmt"
    "unicode/utf8"
)

func main() {
    b := []byte("Hello, 世界")
    valid := utf8.Valid(b)
    fmt.Printf("Byte slice valid: %t\n", valid) // Byte slice valid: true

    s := "Hello, 世界"
    valid = utf8.ValidString(s)
    fmt.Printf("String valid: %t\n", valid) // String valid: true
}
```

Заключение

Пакет `utf8` в Go предоставляет мощные функции для работы с UTF-8 кодировкой, позволяя легко кодировать и декодировать символы, проверять строки на допустимость и подсчитывать количество рун. Эти функции делают обработку текста с символами Unicode удобной и эффективной, что особенно важно для приложений, работающих с многоязычными данными.

Строки под капотом

В Go строки (тип `string`) реализованы как структура, которая содержит указатель на массив байтов и длину строки. Это делает строки эффективными для операций чтения, но их имутабельность означает, что строки не могут быть изменены после создания. Ниже приведен пример того, как строка реализована под капотом и какие аспекты стоит учитывать.

Реализация строк в Go

Строки в Go представлены как структура:


```
type stringStruct struct {
    Data    *byte
    Length int
}
```

На самом деле, Go использует другую структуру данных для хранения строк, но для понимания принципов можно рассмотреть упрощённую модель. В реальном коде это выглядит как:

```
// internal/runtime/string.go (упрощенный вид)
type stringHeader struct {
    Data    uintptr
    Length int
}
```

Принципы реализации

- **Иммутабельность:** Строки в Go иммутабельны, что означает, что их содержимое не может быть изменено после создания. Это достигается тем, что строка содержит указатель на массив байтов, который не может быть изменён напрямую.
- **Ссылочный тип:** Строка в Go является ссылочным типом. Это означает, что при передаче строки в функции передается указатель на её данные, а не копия строки.
- **Хранение данных:** Данные строки хранятся в непрерывной области памяти, которая управляется сборщиком мусора. Это позволяет эффективно работать с строками и минимизировать количество аллокаций.
- **Оптимизация хранения:** Для строк с малым числом символов Go может использовать оптимизированное хранение, чтобы минимизировать накладные расходы.

Пример реализации

В реальном Go коде строка представлена как структура, содержащая указатель на данные и длину строки. Например:

```
type stringStruct struct {
    data    *byte // Указатель на данные строки
    length int  // Длина строки
}
```

Функции, работающие со строками, будут взаимодействовать с этой структурой. Например:

```
func stringLength(s string) int {
    return len(s) // В Go это вызовет встроенную функцию, которая использует
                  // внутреннюю длину строки
}
```

Пример работы с данными строки

Давайте посмотрим, как может выглядеть низкоуровневая работа со строками в Go:

```
package main

import (
    "fmt"
    "unsafe"
)

func main() {
    s := "Hello, World!"

    // Получаем указатель на данные строки и её длину
    header := (*[2]uintptr)(unsafe.Pointer(&s))
    dataPtr := header[0]
    length := int(header[1])

    fmt.Printf("String: %s\n", s)
    fmt.Printf("Data pointer: %x\n", dataPtr)
    fmt.Printf("Length: %d\n", length)
}
```

Важные моменты

- Оптимизация и безопасность: Хотя можно использовать `unsafe` для низкоуровневой работы с указателями, это может привести к проблемам с безопасностью и совместимостью. Рекомендуется использовать стандартные библиотеки и функции для работы со строками.
- Сборка мусора: Поскольку строки являются ссылочными типами и их данные хранятся в куче, сборщик мусора управляет памятью, освобождая неиспользуемые данные.
- Кодировка: Строки в Go используют UTF-8 кодировку, что делает их подходящими для работы с международными символами.

Заключение

Строки в Go реализованы как структуры с указателем на массив байтов и длиной строки. Эта реализация позволяет эффективно работать со строками и поддерживает их иммутабельность. При этом Go предоставляет высокоуровневые функции и методы для работы со строками, что упрощает их использование и минимизирует необходимость низкоуровневого доступа к данным.